

02_Apuntes_Seaborn_Adaptacion_Graficos

May 11, 2026

1 Consejos practicos en Seaborn: Modificar graficos para personalizacion

Una vez visto las librerias que nos permiten realizar visualizacion de datos en Python, el siguiente paso es aprender a modificar y personalizar cada grafico del modo que necesitamos para adecuarlo a nuestros estandares.

En este documento veremos ejemplos y modos de personalizacion para graficos disponibles en la libreria *Seaborn*

Como siempre que trabajamos con esta libreria en este entorno sera necesario activarlo para que pueda interpretarlo:

```
[1]: %matplotlib inline
```

1.1 Configuracion Temas de visualizacion

Las principales caracteriticas que vamos a poder modificar en los graficos para personalizarlas, seran como vimos, los datos y la estructura de los mismos (*long-form*, *wide-form*, *Facets*) que conllevan **modificar aspectos del contenido y forma de los datos**.

Los objetos graficos que devuelven las funciones de Seaborn **constan de los ejes (*axes*) y la representacion de la figura (*figure*)**

Sin embargo, Seaborn tambien permite modificar **el estilo y temas de la visualizacion** que son aspectos que tienen que ver mas **con la apariencia con la que se representan los datos**.

Por defecto Seaborn utiliza los valores de visualizacion de Matplotlib (*rcParams*)

Para modificar el tema o estilo, hacemos uso de la sentencia *set_theme()*

```
sns.set_theme()
```

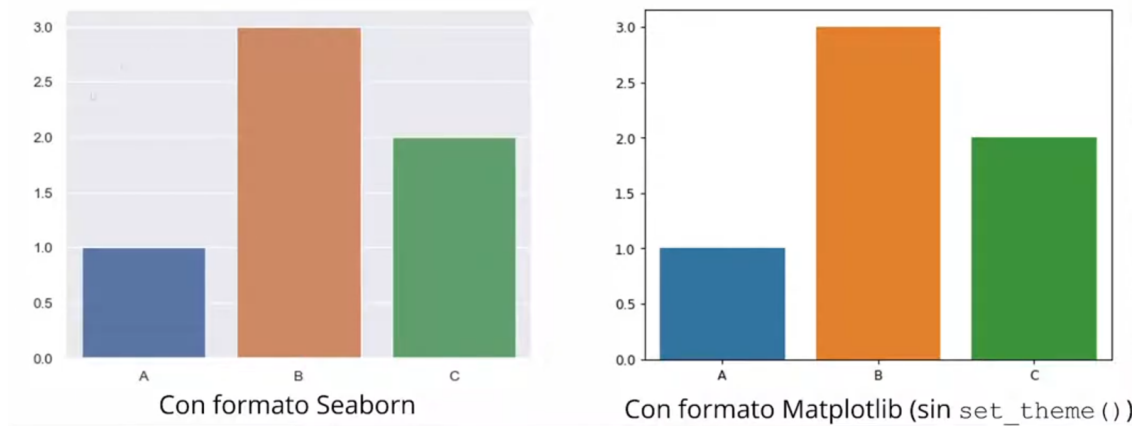
```
sns."metodo para crear grafico"(...)
```

Tiene las siguientes características:

- **Se debe ejecutar antes de generar las figuras**
- Si no se añaden argumentos, **se utilizara por defecto los de Seaborn**
- Permite modificar: *Los colores, el estilo de los ejes, el tamaño y forma de la letra, las líneas,...*

Utilizar *set_theme()* nos permite simplificar la configuracion de graficos y hace que la misma sea mas intuitiva.

A continuacion se ve una imagen de **estilo de Seaborn vs estilo de Matplotlib**, principalmente, rebaja las tonalidades y da un formato “mas limpio” que contextualiza la representacion.



1.1.1 Configuración mediante `set_theme()`

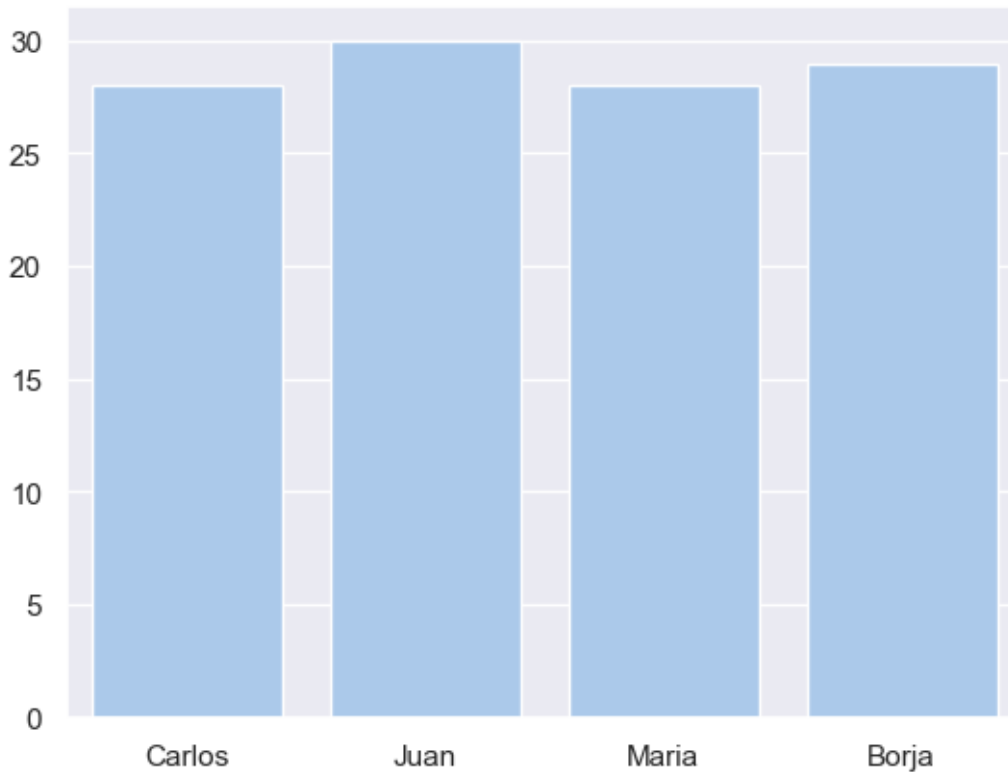
Esta función dispone de los siguientes parámetros básicos que podremos modificar:

- *context* =
- *style* = (de manera genérica)
- *palette* = (colores utilizados)
- *font* = (tipografía)
- *font_scale* = (tamaño de la letra)
- *color_codes* = *True/False*
- *rc* = *None* (si utiliza los rcParams o no)

A continuación hacemos un ejemplo cambiando la paleta de colores.

```
[2]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [28, 30, 28, 29]
sns.set_theme(palette = "pastel")
sns.barplot(x=x, y=y);
```



TIP Importante: Recuerda poner al final de la sentencia que genera el grafico el “;” para que se pueda visualizar.

Sin embargo existen mas parametros que podemos configurar.

Podemos por ejemplo, incluir **marcadores** haciendo uso del parametro *marker* = “...”

Los marcadores son formas de representar cada uno de los puntos de datos que disponemos en el grafico

Entre los marcadores disponibles son: “o”, “x”, “-”

En el siguiente ejemplo, vamos a preparar un grafico que **representa 2 variables con una escala similar** con un elemento diferenciador que permita representar las 2 variables en el mismo grafico (por ejemplo representar *años de x cosa* y *años de y cosa*).

En este caso haremos 2 graficas distintas pero **con elementos de estilo que los diferencien**

Se combinaran los 2 graficos ya **que comparten el mismo eje x**

```
[4]: import matplotlib.pyplot as plt
import seaborn as sns

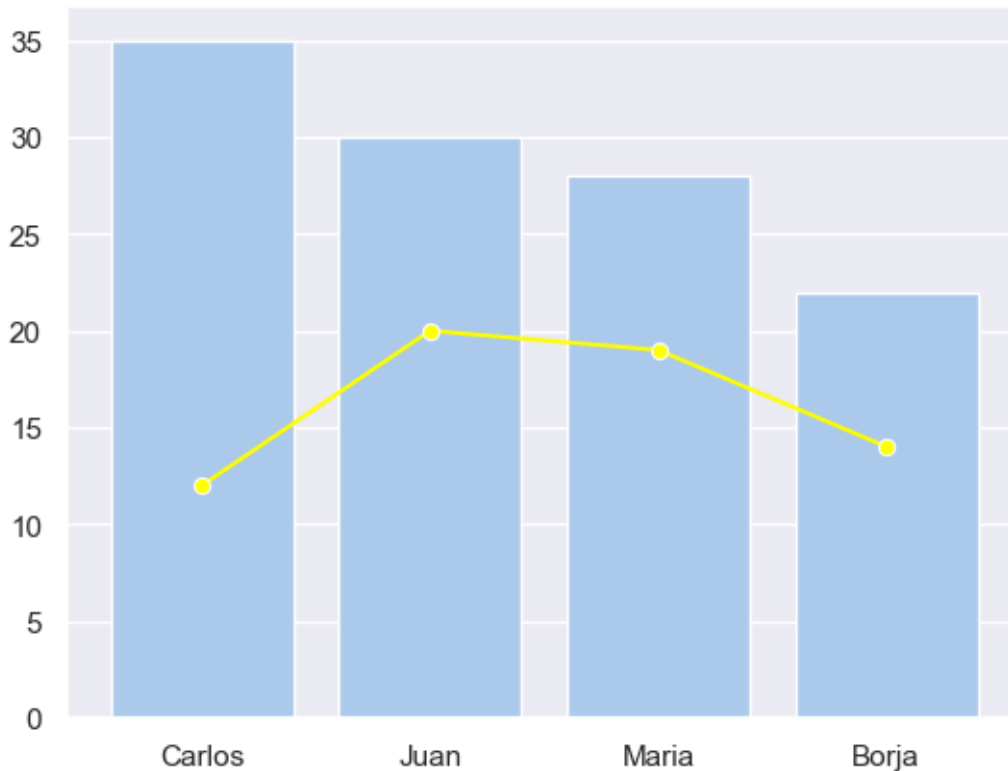
x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
```

```

z= [12,20,19,14]

sns.set_theme(palette = "pastel")
sns.barplot(x=x, y=y)
sns.lineplot(x=x, y=z, color = "yellow", marker = "o");

```



TIP: Cuando se comparte el eje en 2 graficas y estan escritos en el script a continuacion Seaborn los superpone, pero si no, **no van a superponerse y saldrán en el mismo grafico pero como figuras independientes**

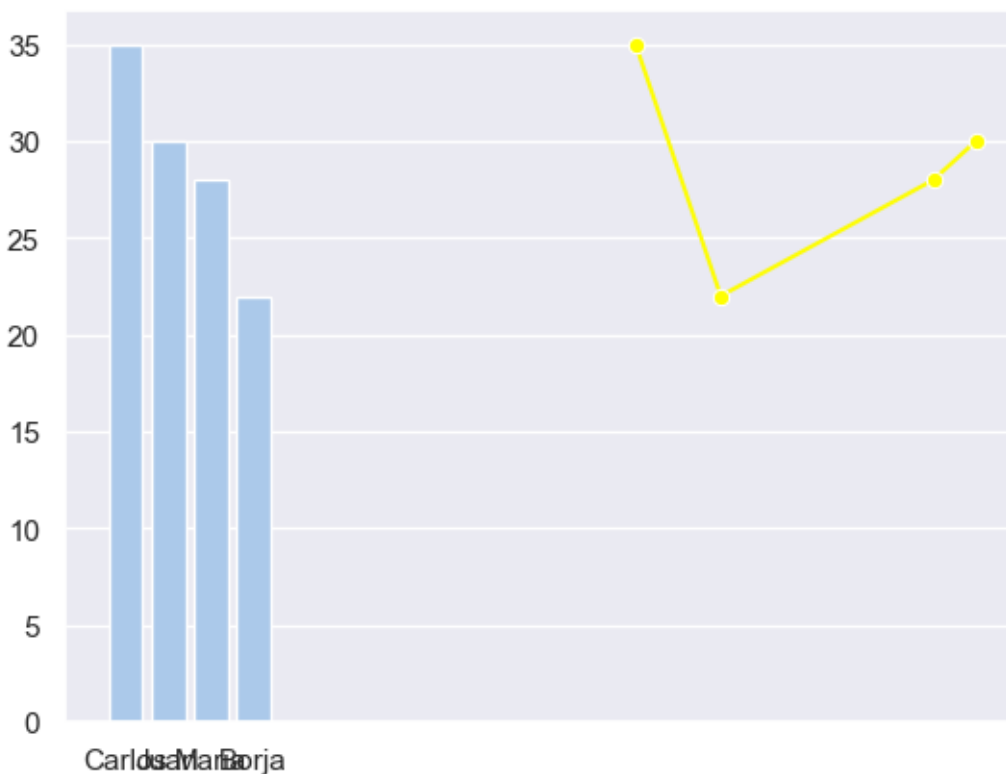
```

[6]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
z = [12,20,19,14]

sns.set_theme(palette = "pastel")
sns.barplot(x=x, y=y)
sns.lineplot(x=z, y=y, color = "yellow", marker = "o");

```



Modificación de parametros basico: Estilo y temas Seaborn incluye cinco temas gráficos predefinidos con distintas configuraciones de estilos para usar directamente con nuestras figuras:

Tema gráfico
darkgrid
whitegrid
dark
white
ticks

Estos temas ajustan elementos gráficos como el color de fondo, líneas de guía, ejes y marcas, leyenda, etc.

Para especificar qué tema queremos aplicar, empleamos la función ***sns.set_style()***.

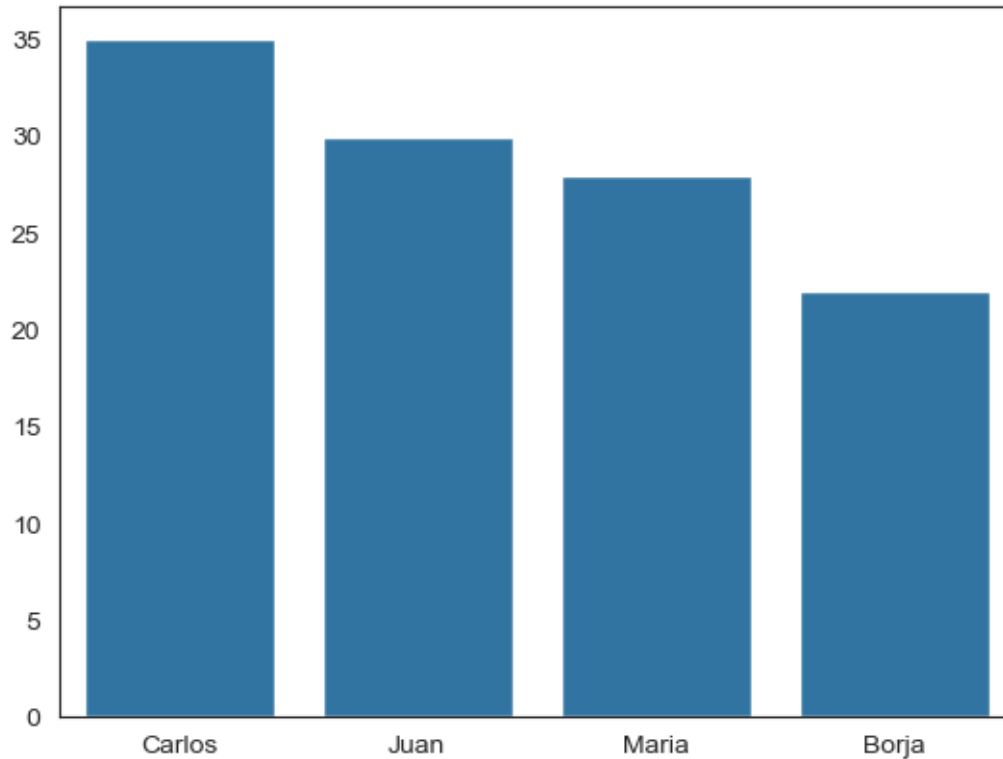
sns.set_style("nombre estilo")

Ademas modificar el estilo permite modificar un listado de parametros mas especificos, para mejorar la personalizacion, se pueden consultar todos haciendo uso de la sentencia ***sns.axes_style()***

```
[1]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]

sns.set_style("white")
sns.barpplot(x=x, y=y);
```



Modificacion de parametros basico: Tamaño de la figura Otro elemento importante es modificar el tamaño de los elementos graficos, lo podremos hacer a traves de `set_size_inches()`

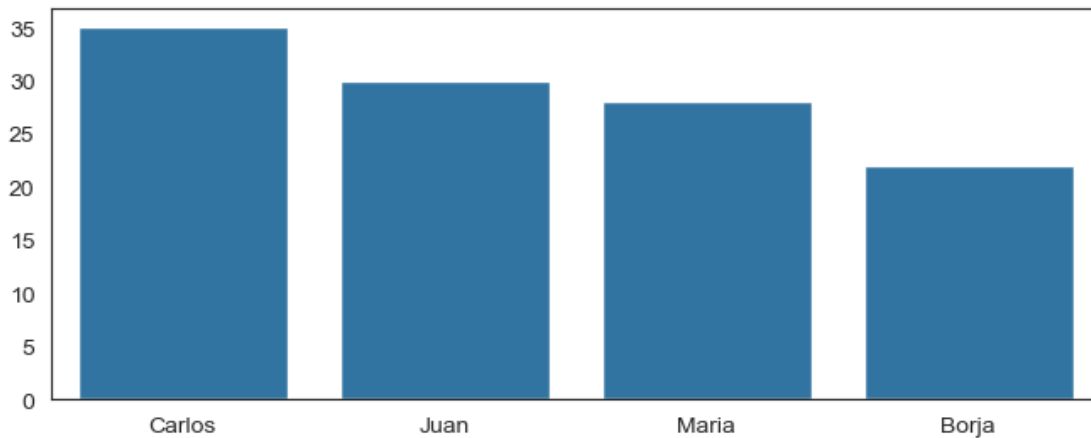
sns."metodo para crear grafico"(...).figure.set_size_inches("ancho deseado", "largo deseado")

```
[7]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]

sns.set_style("white")
```

```
sns.barplot(x=x, y=y).figure.set_size_inches(8,3);
```



Modificacion de parametros basico: Paleta de colores Otro elemento importante es la paleta de colores a elegir, ya que dependiendo del grafico **puede ser adecuado utilizar una u otra para mejorar la visualizacion de los datos**. La sentencia es `sns.color_palette()`:

`sns.“metodo para crear grafico”(…).figure.color_palette()`

Este parametro permite ciertos matices:

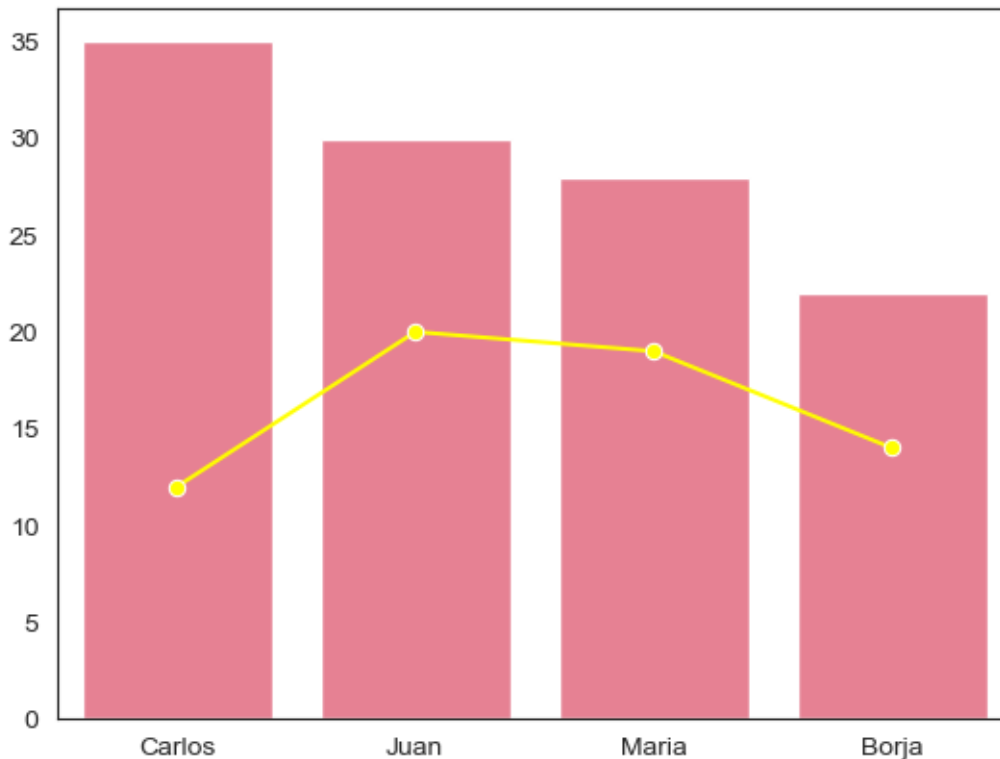
- Si queremos representar *variables cualitativas o valores discretos* es recomendable utilizar el valor `hsl` o `husl`, ya que trata de diferenciar al maximo el tono para cada categoria

`sns.“metodo para crear grafico”(…, .palette = sns.color_palette(hsl/husl, “nº de variables”))`

```
[16]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
z = [12, 20, 19, 14]

sns.set_style("white")
sns.lineplot(x=x, y=z, color = "yellow", marker = "o");
sns.barplot(x=x, y=y, palette = sns.set_palette("husl", 2));
```



- Si queremos personalizar los colores nosotros mismos

sns."metodo para crear grafico"(..., .palette = sns.xkcd_palette(["nombre color",...]))

```
[18]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
z = [12, 20, 19, 14]

sns.set_style("white")
sns.lineplot(x=x, y=z, color = "yellow", marker = "o");
sns.barplot(x=x, y=y, palette = sns.xkcd_palette(["brown"]));
```

C:\Users\insau\AppData\Local\Temp\ipykernel_17740\2067873046.py:10:

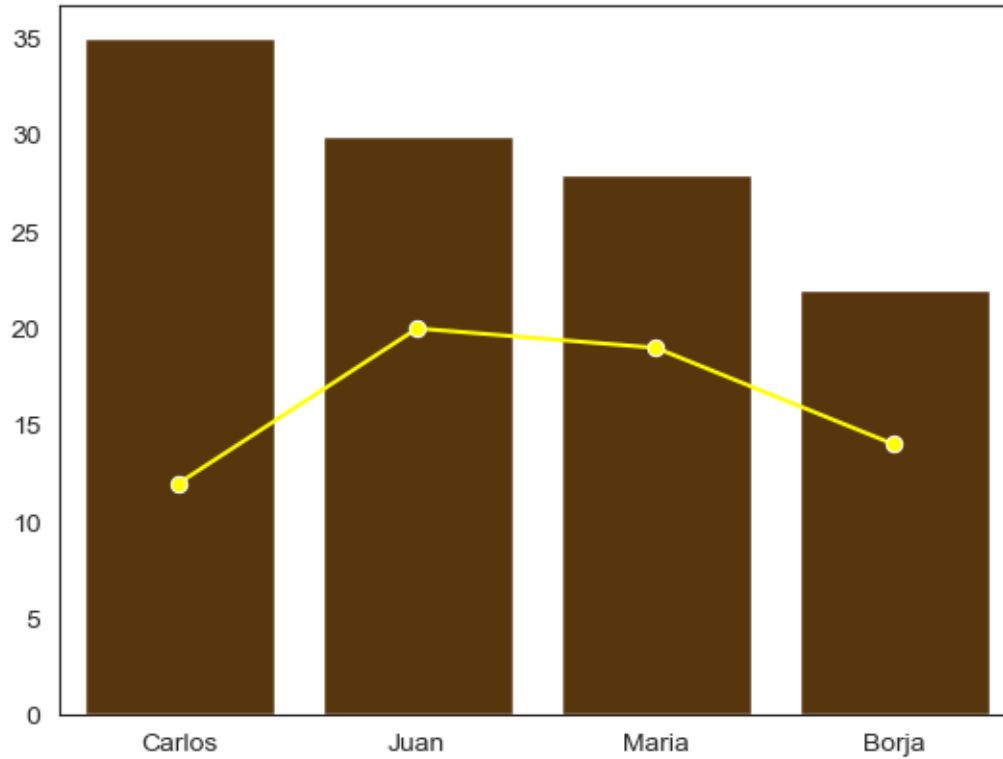
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=x, y=y, palette = sns.xkcd_palette(["brown"]));
```

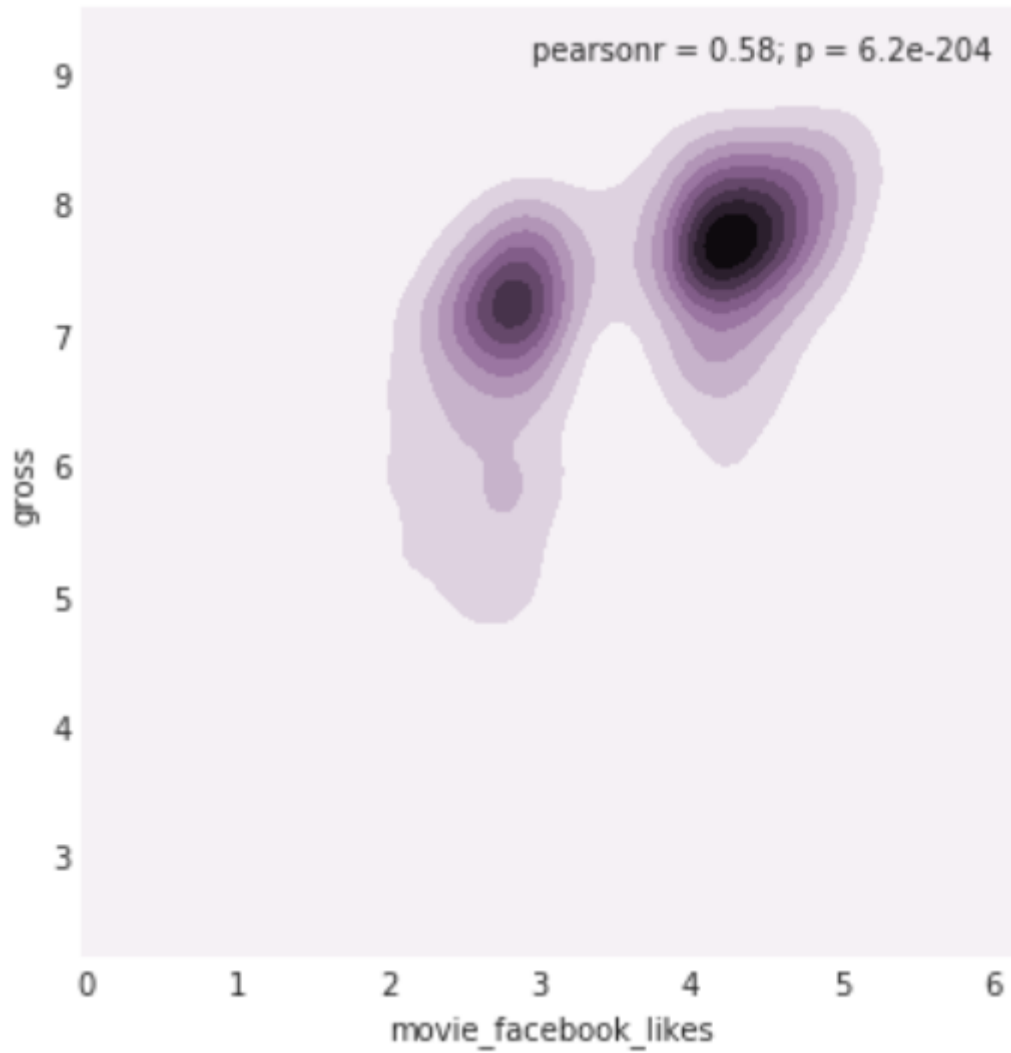

C:\Users\insau\AppData\Local\Temp\ipykernel_17740\2067873046.py:10: UserWarning:
The palette list has fewer values (1) than needed (4) and will cycle, which may
produce an uninterpretable plot.

```
sns.barplot(x=x, y=y, palette = sns.xkcd_palette(["brown"]));
```

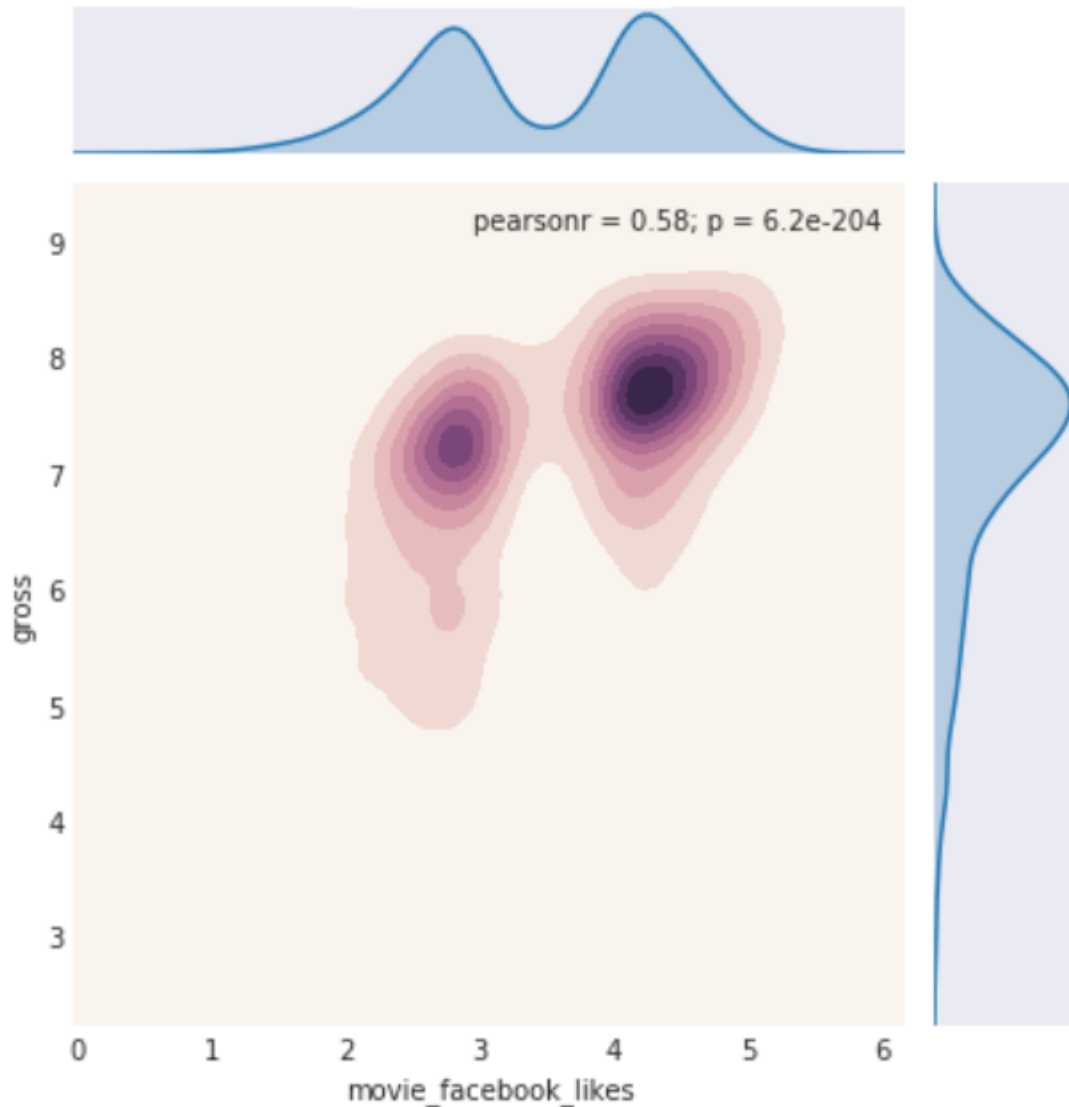


- Si queremos mostrar un variable continua que varia en el tiempo es recomendable utilizar *sns.light_palette()* que partiendo de un color, modifica la saturacion del mismo para mostrar las distribuciones.

sns.light_palette ("nombre color")



- Si queremos incrementar el contraste en una variable que evoluciona en el tiempo es recomendable utilizar `sns.cubehelix_palette()` que permite modificar el tono y el brillo.
`sns.light_palette` (*light* = “graduacion de tono y brillo”)



Modificación de parametros basico: Nombre del Titulo Podemos modificar el titulo de una grafica simple utilizando la sentencia `sns.set_title()`:

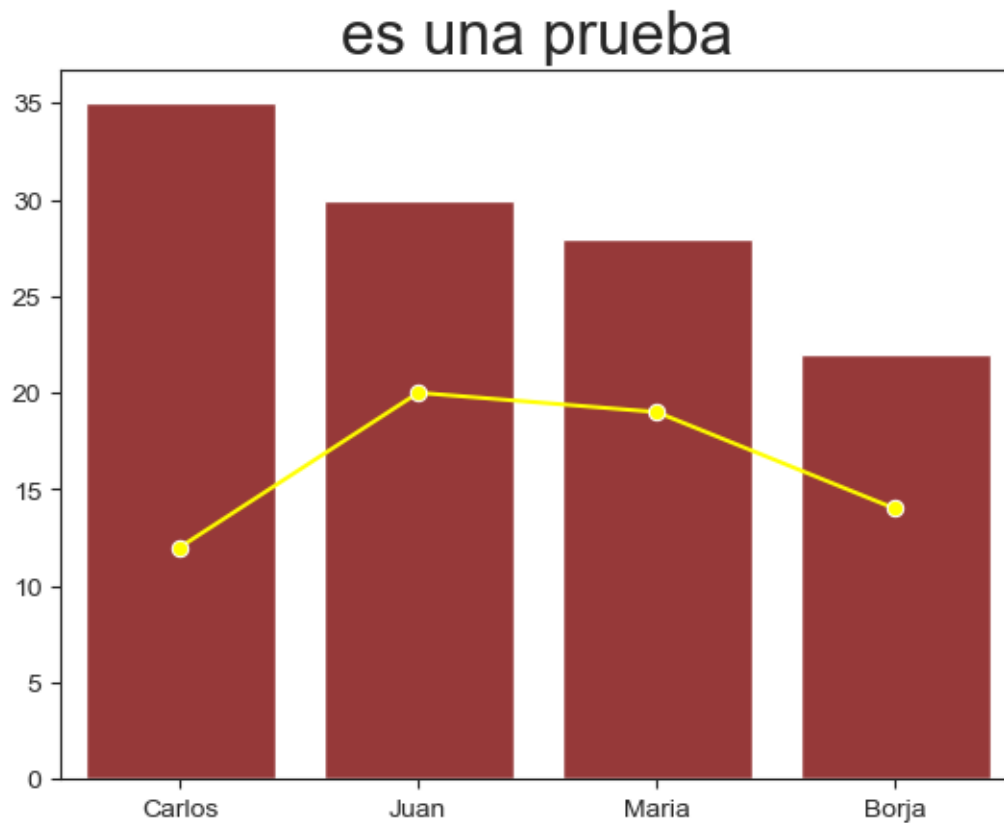
*sns."metodo para crear grafico"(...).set_title("nombre del titulo a poner",
fontsize = "tamaño de letra")*

```
[19]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
z = [12, 20, 19, 14]

sns.set_style("ticks")
sns.lineplot(x=x, y=z, color = "yellow", marker = "o");
```

```
sns.barplot(x=x, y=y).set_title("es una prueba", fontsize = 23);
```



Modificacion de parametros basico: Nombre de las etiquetas de los ejes Podemos modificar las etiquetas de los ejes de una grafica simple utilizando la sentencia *sns.set()* e indicando los valores *xlabel* y *ylabel*:

sns."metodo para crear grafico"(...).set(*xlabel* = "nombre del eje", *ylabel* = "nombre del eje")

```
[20]: import matplotlib.pyplot as plt
import seaborn as sns

x = ["Carlos", "Juan", "Maria", "Borja"]
y = [35, 30, 28, 22]
z = [12, 20, 19, 14]

sns.set_style("ticks")
sns.lineplot(x=x, y=z, color = "yellow", marker = "o");
sns.barplot(x=x, y=y).set(xlabel = "personas", ylabel = "años");
```

